

Python Power Tools: PyFiglet

Welcome to **PyFiglet** — the library that transforms boring plain text into fun ASCII art banners! ✨

◆ What are FIGlet fonts?

- In the early 90's, three people - Frank, Ian and Glenn - came up with a set of ASCII Art fonts, an ASCII art font file spec, and an open source program that would convert text to ASCII art.
- They named their project FIGlet, which is short for "Frank, Ian and Glen's letters".
- FIGlet fonts are the most popular type of ASCII art font.

◆ What problem does it solve?

- Normal console text looks boring 😞
- `pyfiglet` makes CLI output **stylish** with ASCII art 🎨
- Useful for banners, titles, fun project intros.

◆ Real-world use cases

1. Project start banners in CLI tools
2. Adding fun headers to logging systems
3. Styling hackathon/demo scripts to make them stand out 🚀

```
In [1]: # !pip install pyfiglet
```

```
In [2]: # ◆ Small Demo: Import PyFiglet
import pyfiglet
```

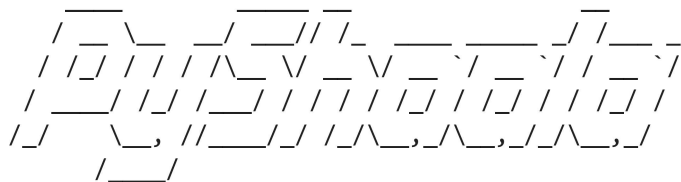
```
banner = pyfiglet.figlet_format("Hello PyShaalaa!")
print(banner)
```



◆ Tips, Tricks & Pitfalls

- You can choose different fonts using `figlet_format(text, font="slant")`
- Not ideal for very long text → output may wrap badly
- Unicode / emojis not supported 🚫
- Works best for **short titles & names**

```
In [3]: # ◆ Try with different fonts
print(pyfiglet.figlet_format("PyShaalaa", font="slant"))
```



```
In [4]: print(pyfiglet.figlet_format("Learning Python", font="banner3-D"))
```



```
In [6]: #  Listing available fonts
from pyfiglet import Figlet
figlet = Figlet()
fonts = figlet.getFonts()
print(fonts)
```

['1943____', '1row', '3-d', '3d-ascii', '3d_diagonal', '3x5', '4max', '4x4_offr', '5lineoblique', '5x7', '5x8', '64f1____', '6x10', '6x9', 'acrobatic', 'advenger', 'alligator', 'alligator2', 'alpha', 'alphabet', 'amc_3_line', 'amc_3_line1', 'amc_aaa01', 'amc_neko', 'amc_razor', 'amc_razor2', 'amc_slash', 'amc_slider', 'amc_thin', 'amc_tubes', 'amc_untitled', 'ansi_regular', 'ansi_shadow', 'aquaplan', 'arrows', 'ascii12', 'ascii9', 'ascii_new_roman', 'ascii____', 'asc_c____', 'assalt_m', 'asslt_m', 'atc_gran', 'atc____', 'avatar', 'a_zooloo', 'b1ff', 'banner', 'banner3-D', 'banner3', 'banner4', 'barbwire', 'basic', 'battlesh', 'battle_s', 'baz_bil', 'bear', 'beer_pub', 'bell', 'benjamin', 'big_g', 'bigascii12', 'bigascii9', 'bigchief', 'bigfig', 'bigmono12', 'bigmono9', 'big_money-ne', 'big_money-nw', 'big_money-se', 'big_money-sw', 'binary', 'block', 'blocks', 'blocky', 'bloody', 'bolger', 'braced', 'bright', 'brite', 'briteb', 'britebi', 'britei', 'broadway', 'broadway_kb', 'bubble', 'bubble_b', 'bubble__', 'bulbhead', 'b_m_200', 'c1____', 'c2____', 'calgphy2', 'caligraphy', 'calvin_s', 'cards', 'catwalk', 'caus_in_', 'char1____', 'char2____', 'char3____', 'char4____', 'character1', 'character2', 'character3', 'character4', 'character5', 'character6', 'character', 'charset_', 'chartr', 'chartri', 'chiseled', 'chunky', 'circle', 'clb6x10', 'clb8x10', 'clb8x8', 'cli8x8', 'clr4x6', 'clr5x10', 'clr5x6', 'clr5x8', 'clr6x10', 'clr6x6', 'clr6x8', 'clr7x10', 'clr7x8', 'clr8x10', 'clr8x8', 'coil_cop', 'coinstak', 'cola', 'colossal', 'computer', 'com_sen_', 'contessa', 'contrast', 'convoy__', 'cosmic', 'cosmike', 'cour', 'courb', 'courbi', 'couri', 'crawford', 'crawford2', 'crazy', 'cricket', 'cursive', 'cyberlarge', 'cybermedium', 'cybersmall', 'cygnet', 'c_ascii_', 'c_consen', 'danc4', 'dancing_font', 'dcs_bfmo', 'decimal', 'deep_str', 'defleppard', 'def_leppard', 'delta_corps_priest_1', 'demo_1__', 'demo_2__', 'demo_m__', 'devilish', 'diamond', 'diet_cola', 'digital', 'do_h', 'doom', 'dos_rebel', 'dotmatrix', 'double', 'double_blocky', 'double_shorts', 'drpepper', 'druid____', 'dwhistled', 'd_dragon', 'ebbs_1__', 'ebbs_2__', 'eca____', 'eftichess', 'eftifont', 'eftipiti', 'eftirobot', 'eftitalic', 'eftiwall', 'eftiwater', 'efti_robot', 'electronic', 'elite', 'emboss', 'emboss2', 'epic', 'etcrvs__', 'e_fist_', 'f15____', 'faces_of', 'fairligh', 'fair_mea', 'fantasy_', 'fbr12__', 'fbr1____', 'fbr2____', 'fbr_stri', 'fbr_tilt', 'fender', 'filter', 'finalass', 'fireing_', 'fire_font-k', 'fire_font-s', 'flipped', 'flower_power', 'flyn_sh', 'four_tops', 'fp1____', 'fp2____', 'fraktur', 'funky_dr', 'fun_face', 'fun_faces', 'future', 'future_1', 'future_2', 'future_3', 'future_4', 'future_5', 'future_6', 'future_7', 'future_8', 'fuzzy', 'gauntlet', 'georgi16', 'georgia11', 'ghost', 'ghost_bo', 'ghoulis', 'glenyn', 'goofy', 'gothic', 'gothic__', 'graceful', 'gradient', 'graffiti', 'grand_p_r', 'greek', 'green_be', 'hades____', 'heart_left', 'heart_right', 'heavy_me', 'helv', 'helvb', 'helvbi', 'helvi', 'henry_3d', 'heroboti', 'hex', 'hieroglyphs', 'high_noo', 'hills____', 'hollywood', 'home_pak', 'horizontal_left', 'horizontal_right', 'house_of', 'hupa_bal', 'hyper____', 'icl-1900', 'impossible', 'inc_raw_', 'invita', 'isometric1', 'isometric2', 'isometric3', 'isometric4', 'italic', 'italics_', 'ivrit', 'jacky', 'jazmine', 'jerusalem', 'joust____', 'js_block_letters', 'js_bracket_letters', 'js_capital_curves', 'js_cursive', 'js_stick_letters', 'katakana', 'kban', 'key_board', 'kgames_i', 'kik_star', 'knob', 'konto', 'konto_slant', 'krak_out', 'larry3d', 'lazy_jon', 'lcd', 'lean', 'letter', 'letters', 'letterw3', 'letter_w', 'lexible_', 'lil_devil', 'line_blocks', 'linux', 'lockergnome', 'madrid', 'mad_nurs', 'magic_ma', 'marquee', 'master_o', 'maxfour', 'mayhem_d', 'mcg____', 'merlin1', 'merlin2', 'mig_ally', 'mike', 'mini', 'mirror', 'mnemonic', 'modern__', 'modular', 'mono12', 'mono9', 'morse', 'morse2', 'moscow', 'mshebre_w210', 'muzzle', 'nancyj-fancy', 'nancyj-improved', 'nancyj-underlined', 'nancyj', 'new_ascii', 'nfi1____', 'nipples', 'notie_ca', 'nfn____', 'nscript', 'ntgreek', 'nvscript', 'o8', 'octal', 'odel_lak', 'ogre', 'ok_beer_', 'old_banne_r', 'os2', 'outrun__', 'pacos_pe', 'pagga', 'panther_', 'patorjk's_cheese', 'patorjk-hex', 'pawn_ins', 'pawp', 'peaks', 'pebbles', 'pepper', 'phonix_', 'platoon2', 'platoon_', 'pod____', 'poison', 'puffy', 'puzzle', 'pyramid', 'p_s_kateb', 'p_s_h_m_', 'r2-d2____', 'radical_', 'rad_phan', 'rad____', 'rainbow_', 'rally_s2', 'rally_sp', 'rammstein', 'rampage_', 'rastan__', 'raw_recu', 'rci____', 'rectangles', 'red_phoenix', 'relief', 'relief2', 'rev', 'ripper!_', 'road_rai', 'rockbox_', 'rok____', 'roman', 'roman____', 'rot13', 'rotated', 'rounded', 'rowancap', 'rozzo', 'runic', 'runyc', 'sans', 'sansb', 'sansbi', 'sansi', 'santa_clara', 'sblood', 'sbook', 'sbookb', 'sbookbi', 'sbooki', 'scrip

t', 'script_', 'serifcap', 'shadow', 'shimrod', 'short', 'skateord', 'skateroc', 'skate_ro', 'sketch_s', 'slant', 'slant_relief', 'slide', 'slscript', 'sl_script', 'small', 'small_caps', 'small_poison', 'small_shadow', 'small_slant', 'smascii12', 'smascii9', 'smblock', 'smbraill', 'smisome1', 'smkeyboard', 'smmono12', 'smmono9', 'smscript', 'smshadow', 'smslant', 'smtengwar', 'sm____', 'soft', 'space_op', 'spc_demo', 'speed', 'spliff', 'stacey', 'stampate', 'stampatello', 'standard', 'starwars', 'star_strips', 'star_war', 'stealth_', 'stellar', 'stencil1', 'stencil2', 'stforek', 'stick_letters', 'stop', 'straight', 'street_s', 'stronger_than_all', 'sub-zero', 'subteran', 'super_te', 'swamp_land', 'swan', 'sweet', 'tanja', 'tav1____', 'taxi____', 'tec1____', 'tecrvs__', 'tec_7000', 'tengwar', 'term', 'test1', 'the_edge', 'thick', 'thin', 'this', 'thorned', 'threepoint', 'ticks', 'ticksslant', 'tiles', 'times', 'timesof1', 'tinker-toy', 'ti_pan_', 'tomahawk', 'tombstone', 'top_duck', 'train', 'trashman', 'trek', 'triad_st', 'ts1____', 'tsalagi', 'tsm____', 'tsn_base', 'tty', 'ttyb', 'tubular', 'twin_cob', 'twisted', 'twopoint', 'type_set', 't_of_ap', 'ucf_fan_', 'ugalympi', 'unarmed_', 'univers', 'usaflag', 'usa_pq__', 'usa____', 'utopia', 'utopiab', 'utopia_bi', 'utopiai', 'varsity', 'vortron_', 'war_of_w', 'wavy', 'weird', 'wet_letter', 'whimsy', 'wideterm', 'wow', 'xbrite', 'xbriteb', 'xbritebi', 'xbritei', 'xchartr', 'xchartri', 'xcour', 'xcourb', 'xcourbi', 'xcouri', 'xhelv', 'xhelvb', 'xhelvbi', 'xhelvi', 'xsans', 'xsansb', 'xsansbi', 'xsansi', 'xsbook', 'xsbookb', 'xsbookbi', 'xsbooki', 'xtime_s', 'xtty', 'xttyb', 'yie-ar__', 'yie_ar_k', 'z-pilot_', 'zig_zag_', 'zone7____']

```
In [7]: print(pyfiglet.figlet_format("PyShaala", font="poison"))
```

[illegible]

```
In [8]: print(pyfiglet.figlet_format("PyShaalaa", font="stop"))
```

[illegible]

```
In [9]: print(pyfiglet.figlet_format("PyShaala", font="sweet"))
```

```
In [10]: print(pyfiglet.figlet_format("PyShaala", font="xsbook"))
```

```
In [11]: print(pyfiglet.figlet_format("PyShaala", font="fire_font-k"))
```

```
(
)\ )      (\ )      (
(()/( ( ( ( ( / (      )      ) \      )
/ ( ) ) \ ) / ( ) ) \ ( ) ( / ( ( ( ) ( / (
( ) ) ( ) / ( ( ) ) ( ( ) \ ) ( ) ) ( ) ) _ ) ( )
| _ \ ) ( ) ) / _ || | ( ) ( ( ) _ ( ( ) _ | | ( ( ) _
| _ / | | | \ _ \ | ' \ / _ ' / _ ' | | / _ ' |
| _ | _ \ , || _ _ / | | | \ _ , _ \ _ , _ | | | \ _ , _
| _ /
```

```
In [12]: print(pyfiglet.figlet_format("Python", font="acrobatic"))
```

```
o_ _o      o      o
<|_ v\      <|> <|>
/ \      <\      < > / >
\o/      o/      |      \o_ _o      o_ _o      \o_ _o
|_ _<|/ <|> <|> o_/_ |_ v\ /v_ v\ |_ |>
|_ < > < > |_ / \ <\ /> <\ / \ / \
<o>      \o o/ |_ \o/ o/ \ / \o/ \o/
|      v\ /v o | <| o o | |
/ \      <\/> <\_ / \ / \ <\_ _/> / \ / \
      /
      o
_/_>
```

```
In [13]: print(pyfiglet.figlet_format("like", font="dancing_font"))
```

```
U |" |      U |" / / \ | _ " | /
U | | u | _ " | | ' / | _ | "
\ | | / _ | | U / | . \ \ u | | _
| _ | U / | | \ u | | \ \ | _ |
// \ \ . , _ | _ | _ , - . , >> \ \ , - . << >>
( _ ) ( _ ) \ _ - ' ' - ( _ / \ . ) ( _ / ( _ ) ( _ )
```

```
In [14]: print(pyfiglet.figlet_format("comment", font="danc4"))
```



```

\0    \0/ \0/ \0/ \0/    0/ '\ /`
 |    _Y  Y   Y   Y   <|  \ /
 / \ _| | / \ / \ / \ / \  X
 \ / _  | | | _ /  \, _| | _ /0\

```

```
In [15]: print(pyfiglet.figlet_format("follow", font="nvscript"))
```

```

,dPYb,          ,dPYb, ,dPYb,
IP'`Yb          IP'`Yb IP'`Yb
I8 8I           I8 8I I8 8I
I8 8'           I8 8' I8 8'
I8 dP ,ggggg,   I8 dP I8 dP ,ggggg, gg gg gg
I8dP dP" "Y8gggI8dP I8dP dP" "Y8gggI8 I8 88bg
I8P i8' ,8I I8P I8P i8' ,8I I8 I8 8I
,d8b,_ ,d8, ,d8' ,d8b,_ ,d8b,_ ,d8, ,d8' ,d8, ,d8, ,8I
PI8"888P"Y8888P" 8P'"Y888P'"Y88P"Y8888P" P'"Y88P'"Y88P"
I8 `8,
I8 `8,
I8 8I
I8 8I
I8, ,8'
"Y8P'

```

```
In [16]: print(pyfiglet.figlet_format("share", font="letter"))
```

```

SSSS H H A RRRR EEEEE
S H H A A R R E
SSS HHHHH AAAAA RRRR EEEE
S H H A A R R E
SSSS H H A A R R EEEEE

```



Practice Exercise for Viewers

👉 Try the following:

1. Print your **name** in 3 different fonts.
2. Print your **YouTube channel name** as a banner.
3. Create a small CLI welcome message using PyFiglet + `input()` (e.g., greet the user).

💡 Share your ASCII art in the comments or repo! 🚀

In [17]: `# !pip install termcolor`

```
In [18]: import pyfiglet
from termcolor import colored

text = "PyShaa1a"
ascii_art = pyfiglet.figlet_format(text, font='fire_font-k')

colors = ["red", "yellow", "green", "cyan", "blue", "magenta"]
result = ""

for i, line in enumerate(ascii_art.splitlines()):
    result += colored(line, colors[i % len(colors)]) + "\n"

print(result)
```

```
(
  (
) \ )      ) \ )      (
(( ) / ( ( ( ) / ( ( ) ) \ )
 / ( ) ) \ ) / ( ) ) \ ( ) / ( ( ) / (
( ) ) ( ) / ( ( ) ) ( ) \ ) ( ) ) ( ) _ ) ( )
| _ \ ) ( ) / _ | | | ( ) ( ) _ ( ) _ | | ( ) _
| _ / | | | \ _ \ | \ / _ \ | \ _ \ | | / _ \ |
| _ \ _ \ | | _ / | | \ _ \ | \ _ \ | | \ _ \ |
| _ /
```

```
In [19]: import pyfiglet
from termcolor import colored

# Text to convert
text = "PyShaa1a"
ascii_art = pyfiglet.figlet_format(text)

# Define rainbow colors
```

```

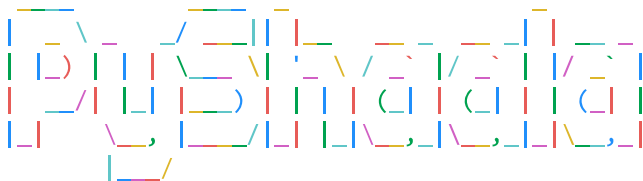
colors = ["red", "yellow", "green", "cyan", "blue", "magenta"]

rainbow_text = ""

# Go character by character
for i, char in enumerate(ascii_art):
    if char.strip(): # only color non-space characters
        rainbow_text += colored(char, colors[i % len(colors)])
    else:
        rainbow_text += char # keep spaces as is

print(rainbow_text)

```



```

In [20]: import pyfiglet
        from PIL import Image, ImageDraw, ImageFont

        # Generate ASCII art
        text = "PyShaaala"
        ascii_art = pyfiglet.figlet_format(text)

        # Font (monospaced is best for ASCII art)
        font = ImageFont.load_default()

        # Split into lines
        lines = ascii_art.split("\n")
        max_width = max([len(line) for line in lines])
        line_height = font.getbbox("A")[3] + 2
        img_height = line_height * len(lines)
        img_width = max_width * 10 # adjust width

        # Create transparent image
        img = Image.new("RGBA", (img_width, img_height), (255, 255, 255, 0))
        draw = ImageDraw.Draw(img)

```

```

# Draw ASCII text in black (or any color)
y = 0
for line in lines:
    draw.text((10, y), line, font=font, fill=(0, 0, 0, 255)) # black text, opaque
    y += line_height

# Save as transparent PNG
img.save("001ascii_art_transparent.png", "PNG")
print("✅ Saved as ascii_art_transparent.png")

```

✅ Saved as ascii_art_transparent.png

```

In [21]: import pyfiglet
from PIL import Image, ImageDraw, ImageFont

def save_rainbow_ascii(text, filename="ascii_rainbow.png", background="transparent"):
    """
    Save ASCII art with rainbow colors as an image (PNG).

    Parameters:
    - text (str): Text to render with figlet
    - filename (str): Output file name
    - background (str | tuple): "transparent", "black", "white" or (R, G, B, A) tuple
    """

    # Generate ASCII art
    ascii_art = pyfiglet.figlet_format(text, font="acrobatic")

    # Define rainbow RGBA colors
    colors = [
        (255, 0, 0, 255),    # red
        (255, 165, 0, 255),  # orange
        (255, 255, 0, 255),  # yellow
        (0, 255, 0, 255),    # green
        (0, 0, 255, 255),    # blue
        (128, 0, 128, 255),  # purple
    ]

    # Set background
    if background == "transparent":
        bg_color = (255, 255, 255, 0)

```

```

elif background == "black":
    bg_color = (0, 0, 0, 255)
elif background == "white":
    bg_color = (255, 255, 255, 255)
elif isinstance(background, tuple):
    bg_color = background
else:
    raise ValueError("Background must be 'transparent', 'black', 'white', or RGBA tuple")

# Use default monospaced font
font = ImageFont.load_default()

# Split into lines
lines = ascii_art.split("\n")
max_width = max([len(line) for line in lines])
line_height = font.getbbox("A")[3] + 2
img_height = line_height * len(lines)
img_width = max_width * 10 # adjust width scaling

# Create image with chosen background
img = Image.new("RGBA", (img_width, img_height), bg_color)
draw = ImageDraw.Draw(img)

# Draw rainbow letters
y = 0
for line in lines:
    x = 10
    for i, ch in enumerate(line):
        if ch.strip(): # only color visible characters
            color = colors[i % len(colors)]
            draw.text((x, y), ch, font=font, fill=color)
            x += 10 # fixed width step
    y += line_height

# Save as PNG
img.save(filename, "PNG")
print(f"✅ Saved as {filename}")

# Example usage:
save_rainbow_ascii("PyShaala", "ascii_transparent.png", background="transparent")
save_rainbow_ascii("PyShaala", "ascii_black.png", background="black")

```

```
save_rainbow_ascii("PyShaala", "ascii_white.png", background="white")  
save_rainbow_ascii("PyShaala", "ascii_custom.png", background=(240, 240, 200, 255)) # pastel yellow
```

- ✓ Saved as ascii_transparent.png
- ✓ Saved as ascii_black.png
- ✓ Saved as ascii_white.png
- ✓ Saved as ascii_custom.png